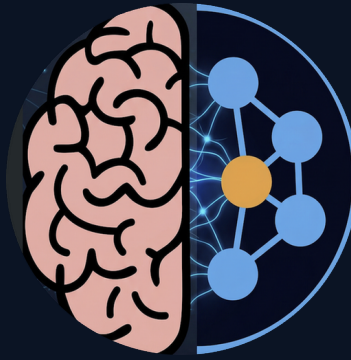




# Parametric Memory

The L2 cache for AI

A verifiable, predictive memory substrate for agents

---

*Technical brief | July 2026*

**[parametric-memory.dev](https://parametric-memory.dev)**

Parametric Memory (MPPM) - Markov-Merkle Predictive Memory

---

## Executive summary

AI agents reason brilliantly over what is in their context window and forget everything the moment it closes. The industry's reflexive fix — retrieval-augmented generation over a vector index — restores *recall*, but recall is only one of the things a production agent needs from memory. An autonomous system operating over long horizons also needs to know **what it knew and when, what tends to come next, when two of its beliefs disagree, and how its facts relate to one another**. A bag of vectors answers none of these.

Parametric Memory is a memory substrate that answers all four — and it does so behind a single Model Context Protocol (MCP) interface, so any MCP-capable AI can use it without a bespoke client. It sits between the model and cold storage the way an L2 cache sits between a CPU and main memory: fast, predictive, and — uniquely — able to prove that what it returned is exactly what it stored.

This brief describes *what* the substrate does and *why the combination matters* for enterprise agents. It deliberately does not describe *how* the mechanisms are implemented; that is where our engineering investment lives, and we are glad to walk qualified evaluators through it under NDA.

---

---

## 1. The problem: capable agents, amnesiac memory

The capability frontier of AI has moved faster than its memory. A modern model can plan a multi-step task, call tools, and write production code — yet between two sessions it retains nothing. Every conversation starts from zero. A user's correction, a design decision, yesterday's root-cause analysis: all of it must be re-derived or re-pasted. The context window is working memory — large, fast, and volatile. What agents lack is *long-term* memory they can trust.

Retrieval-augmented generation is the usual answer: embed documents into vectors, store them in a nearest-neighbour index, retrieve the top matches at query time. For question-answering over static documents, that is often enough. For an autonomous agent, it leaves four questions unanswered:

- **Provenance and integrity** — "*Was this fact actually in memory when I acted on it, and has it been altered since?*" A vector store has no cryptographic answer.
- **Prediction** — "*Given what I just recalled, what do I usually need next?*" Similarity is static and symmetric; it has no notion of sequence.
- **Contradiction** — "*Do two of my stored beliefs disagree?*" Cosine similarity will happily return two mutually exclusive facts as equally relevant, with no signal that they conflict.
- **Structure** — "*How does this fact relate to the others — what does it depend on, supersede, or belong to?*" A bag of vectors has no edges.

These are not luxuries. They are the difference between *document retrieval* and *agent memory*.

---

---

## 2. The idea: memory as an L2 cache for AI

Think of an agent's memory hierarchy the way a computer architect thinks of a CPU's:

| Tier                   | In an AI agent                          | Character                                               |
|------------------------|-----------------------------------------|---------------------------------------------------------|
| L1 / registers         | The context window                      | Tiny, fastest, volatile — lost every session            |
| L2 cache -> this is us | <b>Parametric Memory</b>                | Fast, <b>predictive, verified</b> — the hot working set |
| Main memory / disk     | Vector DBs, knowledge bases, your files | Large, slow, cold storage                               |

Everyone else in the agent-memory space builds *storage* — a place to put things and fetch them back. Parametric Memory builds the **predictive, verifiable tier between the model and that storage**: the layer that has the right context warm *before* the agent asks, and can prove that what it hands over is authentic. Your vector database is main memory. We are the L2 cache.

---

## 3. Four properties, at once, behind one interface

Parametric Memory's name encodes its two load-bearing ideas — it is **Markov** (predictive) and **Merkle** (verifiable) — and the substrate composes four properties that no single existing tool delivers together.

**Verifiable.** Every committed state of memory is an immutable, cryptographically signed snapshot. When an agent recalls a fact, the response can carry a proof binding that fact to a specific, signed version of memory — verifiable by an independent party, offline, without trusting our servers. The substrate also supports *consistency proofs* in the style of Certificate Transparency (RFC 6962): given two versions of memory, it can prove the later one is an append-only extension of the earlier — that history was *added to, not rewritten*. The payoff is a property no vector store has: **every recalled fact comes with a proof of what was known, and when.**

**Predictive.** A passive index answers "what is similar to my query?" Parametric Memory also answers "given what I just recalled, what tends to come next?" It maintains a learned model of the sequences an agent actually follows, so that reinforcing a successful workflow leaves a durable trace the memory can later replay as a prediction. This is what lets memory surface the "*you'll probably need this next*" item that pure similarity would miss.

**Self-curating.** Not everything deserves to be remembered forever. Memory weights decay — and they decay *adaptively*, so a durable rule persists far longer than a transient status, a human-sourced fact outlives an unverified guess, and anything repeatedly used keeps itself alive while the unused fades. Memory, in other words, forgets the way a well-organised mind forgets: preferentially

shedding the ephemeral. No manual gardening required.

**Conflict-aware.** Because knowledge is stored as typed, named units, the substrate can detect contradictions automatically and cheaply. When two stored beliefs disagree, the agent is told *at retrieval time* — and can surface the conflict or resolve it, rather than silently acting on whichever happened to rank higher. This is the primary defence against **stale-truth drift**: the slow accumulation of contradictory beliefs that eventually corrupts any long-lived memory.

**Graph-structured.** Facts do not float in isolation. They are connected by typed, directional edges — depends-on, supersedes, derived-from, belongs-to, and more — that never decay. Retrieval can be reranked by this structure, so an atom richly connected to what you're working on rises, and an atom that has been superseded sinks. The graph is the skeleton; the predictive layer is the metabolism laid over it.

*The mechanisms behind each of these — how decay is tuned per fact, how the ranking fuses lexical, semantic, structural and predictive signals, how the graph and proofs are encoded — are proprietary. This brief is deliberately a description of behaviour, not a blueprint.*

---

## 4. What it feels like to use

Two interaction paths define how an agent uses Parametric Memory.

**The write path.** As the agent encounters durable knowledge — a finding, a decision, a state change, a correction — it commits it in a single call that adds the facts, connects them into the graph, reinforces any successful sequence, and retires anything they supersede, all in one verifiable commit. Knowledge is captured as *it forms*, so an interrupted session loses nothing.

**The read path.** When the agent begins a task, it hands memory its objective and receives, in return, the most relevant, highest-trust, conflict-checked, prediction-augmented slice of everything it has ever known — each item carrying a proof. Mid-task it can recall a specific fact and its likely successors, search, or walk the graph.

The conceptual shift is the whole point: **memory becomes a source the agent queries, not a transcript it re-reads**. The agent does not page through history; it asks a question of a structured, verifiable store and gets back a ranked, proof-carrying answer.

---

## 5. Evidence

We hold our own numbers to an honest standard, including where a simpler baseline wins.

- **Recall is effectively instant, and proof is effectively free.** Median recall latency is sub-millisecond, and verifying a proof costs on the order of tens of microseconds — small

enough that returning a proof with every read is a practical default rather than an expensive opt-in. Sustained throughput runs to several thousand mixed operations per second on a single substrate.

- **Prediction earns its place.** On internal workloads, the predictive layer surfaces the next atom an agent needs a majority of the time (a ~64% hit rate) — recall the memory could not provide by similarity alone.
- **We name where the baseline ties us.** On single-hop keyword lookups, classic lexical retrieval matches us essentially perfectly — memory is not magic on direct keyword questions, and saying so is exactly what makes the harder numbers credible. On multi-hop questions that require connecting facts, structured, predictive memory pulls ahead of both keyword retrieval and a well-crafted long prompt.
- **The economics compound.** The marginal cost of capturing a signal is near zero — one write — but its value compounds, because it contributes to every future inference. Remembering is cheap; forgetting the wrong thing is what's expensive. Each session makes the next one cheaper.

*(Figures describe internal benchmarks on representative workloads and are offered to convey shape and order of magnitude; we are happy to share detailed methodology with evaluators.)*

---

## 6. Why the combination matters

Each property exists in isolation elsewhere. Merkle trees underpin certificate transparency and blockchains. Predictive sequence models are textbook. Knowledge graphs are decades old. Contradiction detection appears in truth-maintenance systems. The contribution is their *composition behind one MCP-native interface* — and the composition is more than additive:

- Verifiability makes prediction **trustworthy** — a predicted-next fact still arrives with a proof of what it is.
- The graph makes conflict detection **richer** — contradictions can be reasoned about along the edges, not merely flagged.
- Prediction makes the graph **dynamic** — it becomes a learned heatmap of how knowledge is *actually used*, not just how it was declared.
- Adaptive decay makes the whole store **self-curating** — it keeps the load-bearing and sheds the ephemeral on its own.

A memory with only similarity search has recall. A memory with all four has *judgement*: it knows what is relevant, what is trustworthy, what is contested, what is connected, and what tends to come next. That is the moat, and it is the reason a competitor cannot reach it by adding one feature.

---

## 7. For the enterprise

**Verifiability is a control, not a nicety.** For agents that take consequential actions — in regulated workflows, agentic commerce, or operations — the question every reviewer eventually asks is *"what did the system actually know when it acted, and can you prove it?"* A log file can be edited. A consistency proof cannot. Parametric Memory turns that question from a liability into a cryptographic answer.

**Designed for record-keeping obligations.** The substrate's append-only, provable history is built to support emerging AI record-keeping requirements — for example, the automatic, traceable event logging contemplated by the EU AI Act's Article 12. It supplies the technical substrate (tamper-evident records with per-record and between-version proofs); the operator still owns the policy decisions — which events count as risk-relevant, and what retention applies. Where personal data and an append-only audit trail are in tension (e.g. erasure requests), the architecture keeps the provable audit plane separate from the deletable personal-data plane.

**Isolation and deployment.** Each customer's memory is an isolated substrate. Entry tiers run in segregated multi-tenant hosting; higher tiers run as dedicated, TLS-fronted instances with no co-tenants. Access is authenticated (OAuth2 and bearer tokens), and the substrate refuses to store content that looks like leaked credentials. Teardowns snapshot before they delete, so data is never removed without a backup.

**Honest about what we don't yet have.** We are not currently SOC 2 or ISO 27001 certified, and we say so; a security-posture briefing is available under NDA. We would rather be trusted than oversell.

---

## 8. Where it pays off

The same substrate serves very different workloads, because the pattern — *ingest the world as typed, connected, verifiable facts, then consult memory instead of re-reading it* — is the same.

- **A research corpus becomes a colleague to consult.** Sources decompose into typed, connected facts; contradictions between papers surface automatically; each citation carries a proof of what the corpus contained at ingestion. The result is an evidence graph, not a pile of passages.
- **A live data feed becomes situational awareness.** Mutable conditions are held as fast-decaying state; discrete occurrences as permanent, auditable events; and the predictive layer answers not just "what is true now?" but "given the current state, what has historically come next?" — with a provable record of what was known at any moment.
- **A codebase becomes persistent structure.** Modules, dependencies, decisions, and bug root-causes become a graph the agent remembers instead of rebuilding every session — and memory learns the workflows the code induces, predicting the migration, the types, and the tests that history says come next.

## 9. Evaluating it

Curiosity is the right response to a claim like "provable memory," and we have built the substrate to be checked rather than taken on faith. Three doors are open to a serious evaluator:

- 1 **Verify a proof yourself.** A signed snapshot can be verified independently — no account required — so you can confirm the core claim before you trust a word of this brief.
- 2 **Run your own benchmark.** We will help you stand up a substrate against a workload that looks like yours.
- 3 **See behind the curtain, under NDA.** The mechanisms this brief withholds — the decay model, the ranking, the codec, the proof construction — are exactly what we walk qualified teams through in a technical deep-dive.

The bottleneck for AI agents is shifting from reasoning to remembering. An agent that cannot retain, trust, and query what it learns is condemned to re-derive its world every session. Parametric Memory is the tier that fixes that — fast enough to feel instant, predictive enough to stay a step ahead, and the only one that can prove what it knew.

---

*Parametric Memory — the L2 cache for AI. Verifiable, predictive agent memory, MCP-native.*  
*[parametric-memory.dev](https://parametric-memory.dev)*